

15-213, 20xx年秋季

数据实验室：位操作 (Data Lab: Manipulating Bits)

发布日期：8月30日，截止日期：9月12日（周三） 晚上11:59

Harry Bovik (bovik@cs.cmu.edu) 是本作业的负责人。

1 简介 (Introduction)

本作业的目的是让您更熟悉整数和浮点数的位级（bit-level）表示。您将通过解决一系列编程“谜题”来实现这一目标。其中许多谜题都显得非常人工化，但您会发现在解决它们的过程中，您将不得不多加思考“位（bits）”是如何运作的。

2 后勤事项 (Logistics)

这是一个个人项目。所有的作业提交均通过电子方式进行。相关的澄清和更正信息将会发布在课程的网页上。

3 获取说明 (Handout Instructions)

站点特定（*SITE-SPECIFIC*）：在此处插入一段解释教师将如何向学生分发 `datalab-handout.tar` 文件的说明。

首先，将 `datalab-handout.tar` 复制到您计划进行作业的 Linux 机器上的一个（受保护的）目录中。然后输入以下命令：

```
unix> tar xvf datalab-handout.tar
```

这将在该目录中解压出许多文件。您**唯一需要修改并提交的文件是 `bits.c`**。

`bits.c` 文件包含 13 个编程谜题的代码框架。您的任务是补全每个函数框架，要求对于整数谜题**只能使用直线型代码（即不能有循环或条件语句）**，并且只能使用数量受限的 C 语言算术和逻辑运算符。具体来说，您只被允许使用以下八个运算符：

```
! ~ & ^ | + << >>
```

其中一些函数会进一步限制这个允许使用的运算符列表。此外，您**不允许使用任何长度超过 8 位（即大于 `0xFF`）的常量**。

有关详细的规则和期望的编码风格的讨论，请参阅 `bits.c` 中的注释。

4 谜题 (The Puzzles)

本节描述了您将在 `bits.c` 中解决的谜题。表 1 按大致的难度顺序（从易到难）列出了这些谜题。“难度分 (Rating)”字段给出了该谜题的难度评级（也是该题的分数），“操作数上限 (Max ops)”字段给出了您在实现每个函数时允许使用的最大运算符数量。有关函数预期行为的更多详细信息，请参阅 `bits.c` 中的注释。您也可以参考 `tests.c` 中的测试函数。这些测试函数被用作参考，用来表达您函数的正确行为，尽管它们自身并不满足您必须遵守的代码编写规则。

名称 (Name)	描述 (Description)	难度分 (Rating)	操作数上限 (Max ops)
<code>bitXor(x,y)</code>	仅使用 <code>&</code> 和 <code>~</code> 计算 <code>x^y</code>	1	14
<code>tmin()</code>	返回最小的二进制补码整数 (TMin)	1	4
<code>isTmax(x)</code>	仅当 <code>x</code> 是最大的补码整数时返回 <code>True</code> (1)	1	10
<code>allOddBits(x)</code>	仅当 <code>x</code> 的所有奇数位都设置为 1 时返回 <code>True</code> (1)	2	12
<code>negate(x)</code>	不使用 <code>-</code> 运算符，计算 <code>-x</code>	2	5
<code>isAsciiDigit(x)</code>	如果 <code>0x30 <= x <= 0x39</code> 则返回 <code>True</code> (1)	3	15
<code>conditional(x,y,z)</code>	实现与 <code>x ? y : z</code> 相同的功能	3	16
<code>isLessOrEqual(x,y)</code>	如果 <code>x <= y</code> 返回 <code>True</code> (1)，否则返回 <code>False</code> (0)	3	24
<code>logicalNeg(x)</code>	不使用 <code>!</code> 运算符，计算逻辑非 <code>!x</code>	4	12
<code>howManyBits(x)</code>	计算用补码表示 <code>x</code> 所需要的最少位数	4	90
<code>floatScale2(uf)</code>	返回浮点数 <code>f</code> 乘以 <code>2</code> (<code>2*f</code>) 的位级等价物	4	30
<code>floatFloat2Int(uf)</code>	返回浮点数 <code>f</code> 转换为整数 (<code>(int)f</code>) 的位级等价物	4	30
<code>floatPower2(x)</code>	对于整数 <code>x</code> ，返回 <code>2.0^x</code> 的位级等价物	4	30

表 1: Data Lab 谜题。对于浮点数谜题，值 `f` 是指具有与无符号整数 `uf` 完全相同位表示的浮点数。

对于浮点数谜题，您将实现一些常见的单精度浮点运算。对于这些谜题，您被允许使用标准的控制结构（如条件语句、循环），并且可以使用 `int` 和 `unsigned` 数据类型，包括任意大小的无符号和整数常量。您不得使用任何联合（unions）、结构（structs）或数组（arrays）。最重要的是，您不得使用任何浮点数据类型。

型、操作或常量。 相反，任何浮点操作数都将作为 `unsigned` 类型传递给函数，任何返回的浮点值也将是 `unsigned` 类型。您的代码应该执行实现指定浮点运算的位操作。

包含在内的程序 `fshow` 可帮助您了解浮点数的结构。要编译 `fshow`，请切换到 `handout` 目录并输入：

```
unix> make
```

您可以使用 `fshow` 来查看任意位模式所表示的浮点数到底是什么：

```
unix> ./fshow 2080374784
Floating point value 2.658455992e+36
Bit Representation 0x7c000000, sign = 0, exponent = f8, fraction = 000000
Normalized. 1.00000000000 X 2^(121)
```

您也可以向 `fshow` 提供十六进制和浮点数值，它会破译它们的位结构。

5 评估 (Evaluation)

您的分数将基于以下分布计算，满分为 67 分：

- 36 正确性分数 (Correctness points)
- 26 性能分数 (Performance points)
- 5 风格分数 (Style points)

正确性分数：您必须解决的谜题被赋予了 1 到 4 的难度评级，它们的加权总和为 36 分。我们将使用 `btest` 程序来评估您的函数（该程序将在下一节描述）。如果您的一道谜题通过了 `btest` 执行的所有测试，您将获得该谜题的满分，否则没有分数（0分）。

性能分数：我们在本课程现阶段的主要关注点是您能得到正确答案。但是，我们希望向您灌输一种“让代码尽可能简短和简单”的意识。此外，某些谜题可以用蛮力（brute force）解决，但我们希望您更聪明。因此，对于每个函数，我们都设定了您允许使用的最大运算符数量。这个限制非常宽容，仅用于抓住极其低效的解决方案。对于每个正确且满足运算符数量限制的函数，您将获得 2 分。

风格分数：最后，我们保留了 5 分用于对您解决方案的风格和注释进行主观评估。您的解决方案应尽可能清晰明了。您的注释应该具有参考价值，但不必过于冗长。

自动评分您的工作 (Autograding your work)

我们在 `handout` 目录中包含了一些自动评分工具 —— `btest`，`dlc`，和 `driver.pl` —— 以帮助您检查工作的正确性。

- **btest:** 此程序检查 `bits.c` 中函数的功能正确性。要构建和使用它，请键入以下两个命令：

```
unix> make
unix> ./btest
```

请注意，每次修改 `bits.c` 文件时，都必须重新构建 `btest`。您会发现一次只处理一个函数，边写边测试会很有帮助。您可以使用 `-f` 标志指示 `btest` 仅测试单个函数：

```
unix> ./btest -f bitXor
```

您可以使用选项标志 `-1`，`-2`，和 `-3` 为其提供特定的函数参数：

```
unix> ./btest -f bitXor -1 4 -2 5
```

查看文件 `README` 以获取有关运行 `btest` 程序的更多文档。

- **dlc**: 这是来自 MIT CILK 小组的 ANSI C 编译器的修改版本，您可以用来检查每个谜题是否遵守了编码规则。典型用法是：

```
unix> ./dlc bits.c
```

该程序会静默运行，除非它检测到问题，例如非法运算符、太多运算符，或在整数谜题中使用了非直线代码（如分支循环）。使用 `-e` 开关运行：

```
unix> ./dlc -e bits.c
```

这会使 `dlc` 打印出每个函数具体使用的运算符数量统计。键入 `./dlc -help` 以获取命令行选项的列表。

- **driver.pl**: 这是一个驱动程序脚本，它使用 `btest` 和 `dlc` 来自动计算您解决方案的正确性和性能得分。它不接受任何参数：

```
unix> ./driver.pl
```

您的授课教师将使用 `driver.pl` 来评估您的最终解决方案。

6 提交说明 (Handin Instructions)

站点特定 (*SITE-SPECIFIC*)：在此处插入文本，告诉每个学生如何在您的学校提交他们的 `bits.c` 解决方案文件。

7 建议 (Advice)

- 不要在您的 `bits.c` 文件中包含 `<stdio.h>` 头文件，因为它会使 `dlc` 感到困惑并导致一些不直观的错误消息。即使不包含 `<stdio.h>` 头文件，您仍然可以在 `bits.c` 文件中使用 `printf` 进行调试，虽然 `gcc` 会打印一条警告，但您可以完全忽略它。
- `dlc` 程序强制执行比 C++ 或一般 `gcc` 强制执行的更严格的 C 声明格式。特别是，**任何声明必须出现在代码块（即封闭在花括号内的区域）中的任何非声明语句之前**。例如，它会对以下代码报错：

```
int foo(int x)
{
    int a = x;
    a *= 3;      /* Statement that is not a declaration (非声明语句) */
    int b = a;   /* ERROR: Declaration not allowed here (错误：此处不允许声明) */
}
*/
```

8 “打败教授”竞赛 (The "Beat the Prof" Contest)

为了好玩，我们提供了一个可选的“打败教授”竞赛，允许您与其他学生和讲师竞争开发最高效的谜题解决方案。目标是使用最少数量的运算符来解决每个 Data Lab 谜题。对于每个谜题，运算符数量匹配或击败讲师代码的学生就是赢家！

要向竞赛提交您的作品，请输入：

```
unix> ./driver.pl -u 'Your Nickname'
```

昵称 (Nickname) 被限制在 35 个字符以内，可以包含字母数字、撇号、逗号、句号、破折号、下划线和与号。您可以尽可能频繁地提交。您最近一次的提交将出现在实时记分板上，并且仅由您的昵称标识。您可以通过将浏览器指向以下地址来查看记分板：

```
http://$SERVER_NAME:$REQUESTD_PORT
```

站点特定 (SITE-SPECIFIC)：将 `$SERVER_NAME` 和 `$REQUESTD_PORT` 替换为您在 `./contest/Contest.pm` 文件中设置的值。